

Sem Factory (Errado)

O cliente precisa saber detalhes de implementação para criar os objetos.

Sem Factory (Errado)

O cliente precisa saber detalhes de implementação para criar os objetos.

```
public class Car
{
    public string Model { get; set; }
}

public class Bike
{
    public string Type { get; set; }
}

// Código cliente
Car car = new Car { Model = "Sedan" };
Bike bike = new Bike { Type = "Mountain" };
```

Problemas:

- O cliente precisa instanciar diretamente as classes.
- A lógica de criação de objetos fica espalhada no código.

Com Factory (Certo)

Centralizamos a lógica de criação em uma fábrica.

```
// Definição de um tipo base
public interface IVehicle
{
    void Drive();
}

// Implementações concretas
public class Car : IVehicle
{
    public void Drive()
    {
        Console.WriteLine("Driving a car!");
    }
}

public class Bike : IVehicle
{
    public void Drive()
    {
        Console.WriteLine("Riding a bike!");
    }
}

// Fábrica para criar objetos
public class VehicleFactory
{
    public static IVehicle CreateVehicle(string vehicleType)
    {
        return vehicleType.ToLower() switch
        {
            "car" => new Car(),
            "bike" => new Bike(),
            _ => throw new ArgumentException("Invalid vehicle type")
        };
    }
}

// Código cliente
IVehicle vehicle = VehicleFactory.CreateVehicle("car");
vehicle.Drive(); // Output: "Driving a car!"

vehicle = VehicleFactory.CreateVehicle("bike");
vehicle.Drive(); // Output: "Riding a bike!"
```

Vantagens do Factory Pattern

1. **Centralização da lógica de criação:** Todo o código de criação de objetos fica em um único lugar.
2. **Flexibilidade:** É fácil adicionar novos tipos de objetos sem alterar o código cliente.
3. **Desacoplamento:** O cliente não precisa saber detalhes das classes concretas.

Quando evitar?

- Quando a lógica de criação é simples e não há necessidade de abstração.
- Em sistemas pequenos, o uso do padrão pode adicionar complexidade desnecessária.